NEURAL & OPTIMIZATION BUDGETING PROJECT DOCUMENTATION

Project Description

Neural Budgeting is a modern web application that combines **Artificial Intelligence** (Neural Networks) and **Operations Research** (Linear Programming) to automate personal finance management. It allows users to input their financial data in natural language, automatically categorizes those expenses, and optimally allocates disposable income across multiple savings goals based on urgency and priority.

Project Structure & Files

The project is divided into three main components: **Frontend**, **Backend**, and **Machine Learning Pipeline**.

Frontend (\frontend)

The user interface built with React, TypeScript, Vite, and shadcn/ui.

Directory: `src/pages/`
Contains the main application views:

- `SmartInput.tsx` : The interface where users type natural language text. It sends text to the backend to be parsed by the neural network into structured incomes, expenses, and goals.
- `Allocation.tsx` : Displays the optimization results, including income sources and mathematically optimized goal funding.
- `Ledger.tsx` : A comprehensive list of categorized expenses and dynamic charts visualizing spending breakdowns over time.
- `About.tsx` : The project information and team page.

Directory:
Contains reusable UI components (Buttons, Cards, Date Pickers, Charts) styled via Tailwind CSS and built on Radix UI.

Directory:
Contains utility functions, predefined category definitions, and which manages English/Arabic language switching and RTL support.

Backend (\backend)

The REST API server built with FastAPI (Python) that handles data processing and optimization.

- `main.py` : The entry point for the FastAPI server. It defines all API endpoints (e.g., ,) and maintains the in-memory data store for transactions and goals.
- `models.py` : Contains Pydantic data models used to validate incoming requests and structure the API responses.
- `smart_parser.py` : Wraps the trained ML model. It loads the and provides the logic to extract dates, monetary amounts, and classify the text into budgeting categories.
- `optimizer.py` : The core optimization engine that uses Linear Programming to distribute disposable income across the user's goals.

Machine Learning Pipeline (\ml_pipeline)

The scripts and assets used to generate data and train the AI categorization model.

- `generate_data.py` : A Python script that generates thousands of synthetic financial descriptions using the `Faker` library.
- `transactions.csv` : The output dataset created by `generate_data.py.`
- `train_nlp.py` : The script that reads the dataset, defines the Neural Network architecture, trains the model, and exports it.
- `expense_classifier.pkl` : The serialized (saved) pipeline containing both the trained neural network and the text vectorizer, which is loaded by the backend.

The Neural Network (\MLPClassifier)

The project uses a **Multi-Layer Perceptron (MLP) Neural Network** from the `scikit-learn` library (`MLPClassifier`).

Purpose:
To understand natural language inputs (like "I spent $50 at Walmart today") and classify them into predefined budgeting categories (e.g., Food, Housing, Transport).

Architecture:

Text Vectorization: A `TfidfVectorizer` first extracts 1-gram and 2-gram word features from the text, converting human sentences into numerical feature vectors (up to 10,000 features).

Hidden Layers:
The neural network has two hidden dense layers:

- First layer: 128 neurons with `ReLU` activation function
- Second layer: 64 neurons with `ReLU` activation function

Both layers learn complex, non-linear patterns in the transaction descriptions.

Output Layer:
A `Softmax` layer maps the final outputs to probability scores across the possible budgeting categories.

Optimizer:
The model is trained using the Adam optimizer for up to 500 epochs, with early stopping enabled to halt training when validation accuracy plateaus, preventing overfitting.

Training Data

The data used to train the neural network is completely synthetic, created using the `ml_pipeline/generate_data.py` script.

Data Location:
Stored locally within the project at: `ml_pipeline/ transactions.csv.`

Data Generation Process:
The script uses the `Faker` Python library to procedurally generate 15,000 highly realistic transaction descriptions.

The generation process:

- Randomly combines various sentence templates (e.g., "I spent ${amount} on {desc}")
- Uses an extensive list of real-world stores and items specific to each category
  - Example: "Uber trip" for Transport category
  - Example: "Netflix" for Subscriptions category
- Injects noise including dates, transaction IDs, and cities
- Simulates messy real-world bank statements and natural typing habits

Budget Optimization Technique

The project employs **Linear Programming (LP)**, a mathematical optimization technique, to determine the absolute best way to distribute disposable income across various savings goals.

Engine Details:

- Location: `backend/optimizer.py`
- Library: Python `PuLP` library

- Solver: `CBC` (Coin-or branch and cut) solver

How It Works:

STEP 1: Disposable Income Calculation
The engine calculates the user's total monthly income and subtracts all essential and recurring expenses to find the "disposable income".

STEP 2: Objective Function
The engine's mathematical goal is to maximize the total weighted value of funded goals. The "weight" (importance) of each goal is calculated by multiplying:

- Priority: A user-defined ranking
- Urgency: A multiplier based on the months remaining until the goal's deadline (closer deadlines receive a higher multiplier)
- Favorite Bonus: Goals marked as a favorite receive a massive 10x multiplier to ensure they get funded first

STEP 3: Constraints

- The sum of all allocated funds cannot exceed the total disposable income
- Each individual goal has dynamic upper and lower bounds to ensure it receives a reasonable portion without starving other goals

STEP 4: Resolution
The CBC solver evaluates all possible distributions simultaneously and guarantees finding the mathematically optimal dollar amount to allocate to each goal.

Output:

- Personalized allocation plan for each goal
- Calculated "safe daily spend" limit for any remaining unallocated funds

Summary

This documentation provides a comprehensive technical overview of the Neural Budgeting project, detailing its structure, key components, and underlying technologies. It covers the frontend, backend, machine learning pipeline, neural network architecture, training data, and budget optimization technique, offering a complete reference for developers and stakeholders.